

Nichelle Graf

Team: Root Matrix

A02 Comparative Analysis of ML & Deep Learning Tools

Individual comparison topics: TensorFlow vs. PyTorch; PyTorch vs. fastai

ITAI 2376 – Deep Learning in AI

Reflection Journal

This assignment helped me understand that choosing a deep learning framework is not simply a matter of identifying which tool is “best.” The more useful question is what kind of workflow, level of control, and deployment environment a project requires.

In comparing TensorFlow and PyTorch, I learned that many of the common distinctions between them are becoming outdated. TensorFlow historically emphasized large-scale deployment and distributed computation, while PyTorch became known for a more flexible, Python-native development experience. However, both frameworks have expanded considerably. PyTorch now has mature production and distributed-training capabilities, while TensorFlow offers a more accessible high-level workflow through Keras. This showed me that framework comparisons need to consider how tools have evolved rather than relying on older assumptions.

The PyTorch and fastai comparison was even more interesting because fastai is built on PyTorch rather than being a completely separate competing framework. I learned that the real difference is largely one of abstraction. fastai simplifies common deep learning workflows and reduces repetitive coding, while direct PyTorch development gives developers more control over model architecture, training logic, and data processing.

The medical-imaging examples made this distinction especially clear. fastai’s chest X-ray workflow demonstrated how a high-level API can make a specialized application accessible with less low-level setup. In contrast, PathAI’s use of PyTorch for extremely large pathology images showed why lower-level control becomes important when a problem requires custom sampling, multiple-instance learning, specialized architectures, and interpretability.

Overall, this assignment changed how I think about deep learning tools. I now see frameworks less as isolated products competing for a single “winner” and more as layers within a broader development ecosystem. The appropriate choice depends on the problem, the team’s expertise, the amount of customization required, and how quickly a solution needs to move from experimentation to production.

Building the notebook helped me understand the framework differences more clearly than research alone. I could see directly how TensorFlow/Keras simplifies training through high-level methods such as `fit()`, while PyTorch exposes the training loop, gradient calculation, and optimizer updates more explicitly. The fastai demonstration made the abstraction difference even more obvious because much of the setup and training process was handled through higher-level APIs.

I also learned an important lesson about dataset selection when an early fastai subset produced an unrealistic 100% accuracy because the sample was not balanced across classes. Correcting the dataset reinforced the importance of validating data before interpreting model results.